UNIFEOB

CENTRO UNIVERSITÁRIO DA FUNDAÇÃO DE ENSINO OCTÁVIO BASTOS
ESCOLA DE NEGÓCIOS

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
CIÊNCIAS DA COMPUTAÇÃO

Configuração inicial e primeiros testes em nuvem – Entrega Prof. Rodrigo Marudi 13/10/2026

ESTUDANTES:

25000151 - Guilherme P. da Rosa Santi

25001176 - Henrique de Oliveira Molinari

25000019 - João Augusto de Freitas

25000795 - Kauan Leander Leandrini

São João da Boa Vista–SP

2026

1. Contexto

Esta entrega parcial, proposta pelo Prof. Rodrigo Marudi em 06/10/2026, pede que a gente monte a estrutura básica de um ambiente de nuvem com suporte a aplicações Node.js, mostre os serviços criados e configurados (incluindo um banco de dados acessível) e registre logs e prints da conexão entre os componentes do sistema.

O SOUFER Tools tem três partes: o front-end (React), a API (Node.js com Express) e o banco PostgreSQL. As três já estão no ar e em uso real pela manutenção da Soufer, então o que mostramos aqui é o ambiente onde o sistema está rodando de verdade, e não um laboratório separado.

Um ponto importante: **nós não usamos a AWS nem a Azure nesta entrega**. Ainda não temos créditos nessas plataformas e o acesso ao laboratório de projetos da AWS não foi liberado para o nosso grupo. Como o sistema já está em produção, com dados reais, achamos arriscado migrar tudo às pressas para uma conta provisória (troca de endereço, risco de perder dados, sistema fora do ar). Por isso montamos o ambiente em um servidor nosso, como solução paliativa, usando ferramentas que reproduzem na prática os mesmos conceitos vistos nas aulas: máquina Linux, contêineres, banco PostgreSQL separado, proxy reverso com HTTPS, deploy automático e monitoramento. Assim que o sistema estabilizar, vamos refazer a implantação também na AWS e na Azure (seção 5).

2. O que foi feito

2.1 Servidor e plataforma

Usamos o **Dokploy** (v0.30.2), uma plataforma open source de deploy que roda em cima de Docker e do Traefik, instalada no nosso servidor (7,65 GiB de memória e 112 GB de disco). Cada serviço roda em um contêiner próprio, o Traefik recebe as requisições por domínio e cuida do HTTPS, e o código vem direto do repositório do GitHub, com build automático pelo Nixpacks. No projeto **Soufer Tools** temos 6 serviços (4 aplicações e 2 bancos), todos rodando e sem erro.

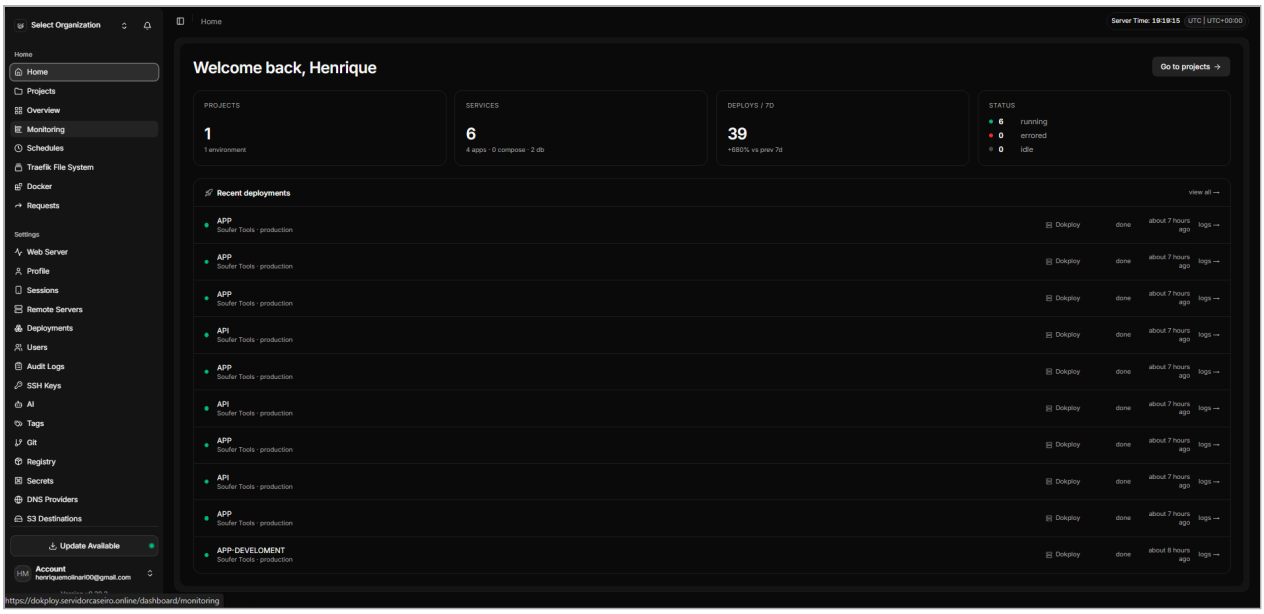


Figura 1 – Painel do Dokploy: 6 serviços rodando, 0 com erro e 39 deploys na última semana.

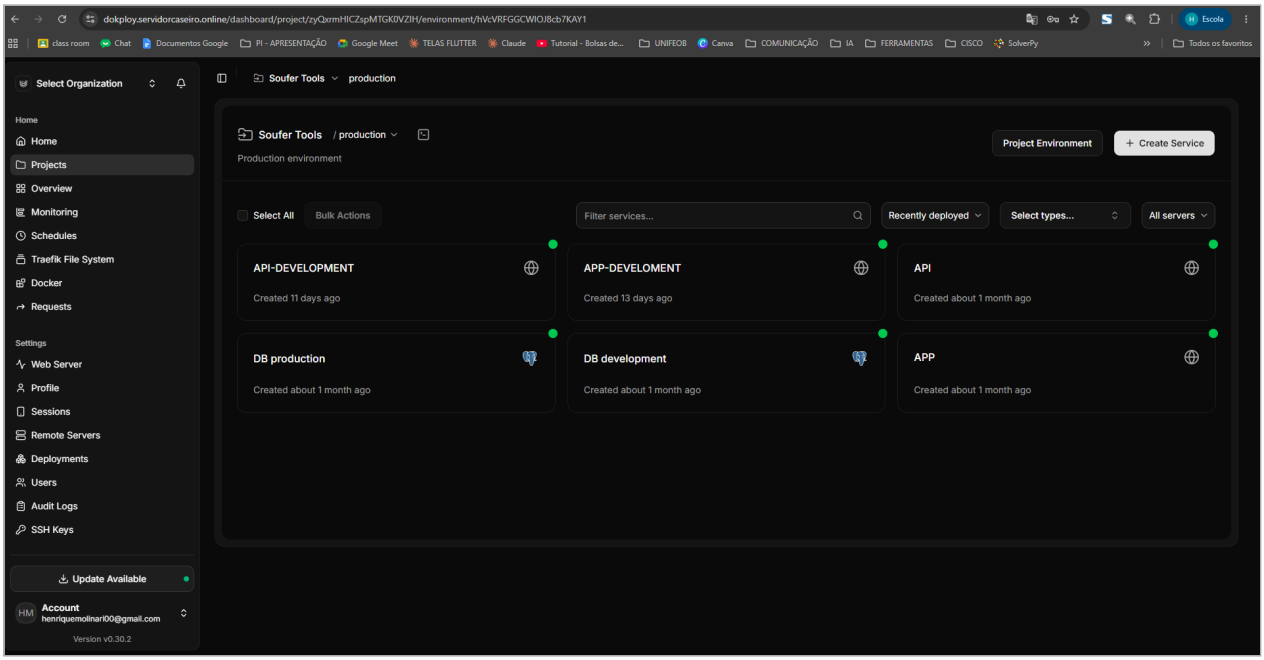


Figura 2 – Projeto Soufer Tools: API, APP e banco, em produção e em desenvolvimento.

Temos dois ambientes no mesmo servidor, cada um com seu banco, sua API e seu front:

- **Produção** (branch `main`): front em `soufer-tools.servidorcaseiro.online`, API em `api-soufer.servidorcaseiro.online` e banco `soufer_prod`.
- **Desenvolvimento** (branch `development`): front em `soufer-tools-development.servidorcaseiro.online`, API em `api-dev-soufer.servidorcaseiro.online` e banco `soufer_dev`.

2.2 Banco de dados

O PostgreSQL foi criado pelo Dokploy em um contêiner próprio. A API conversa com ele pela rede interna do Docker, usando o nome do contêiner como host (porta 5432), e a senha fica guardada no serviço, escondida no painel. As tabelas foram criadas por migrations versionadas (pasta `api/db/migrations` do repositório), e a tabela `schema_migrations` registra quais já foram aplicadas.

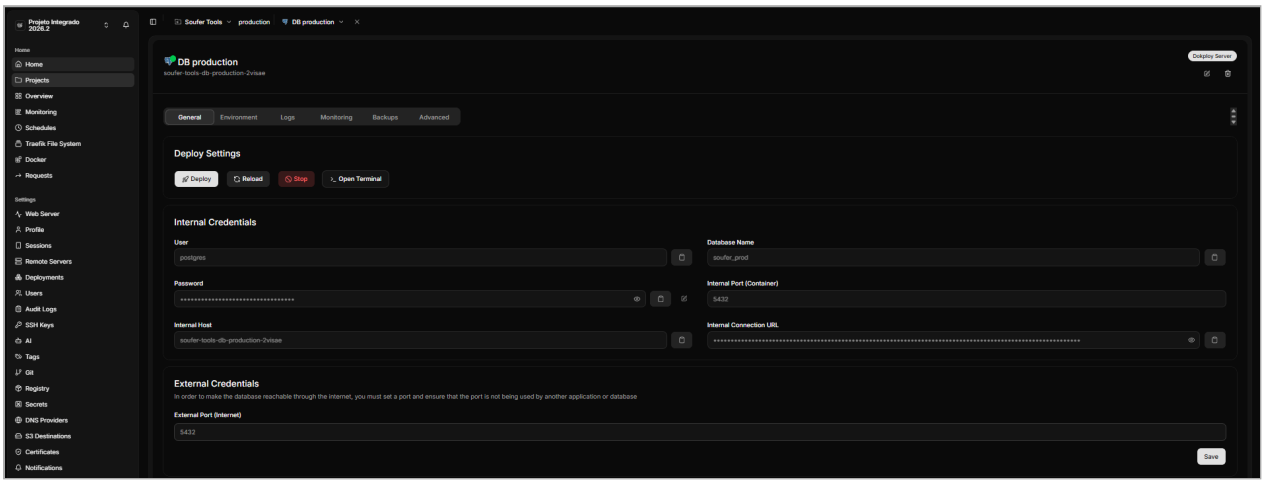


Figura 3 – Serviço do banco de produção: usuário, banco `soufer_prod`, host interno e porta.

2.3 API Node.js

A API foi criada a partir do repositório do GitHub (branch `main`), com build Nixpacks e deploy automático ligado, e publicada no domínio `api-soufer.servidorcaseiro.online`, porta 3000. Toda a configuração (porta, ambiente, origem permitida no CORS, dados do banco, segredo do JWT) fica em variáveis de ambiente do serviço, nada de segredo no código. Na Figura 6 escondemos os valores.

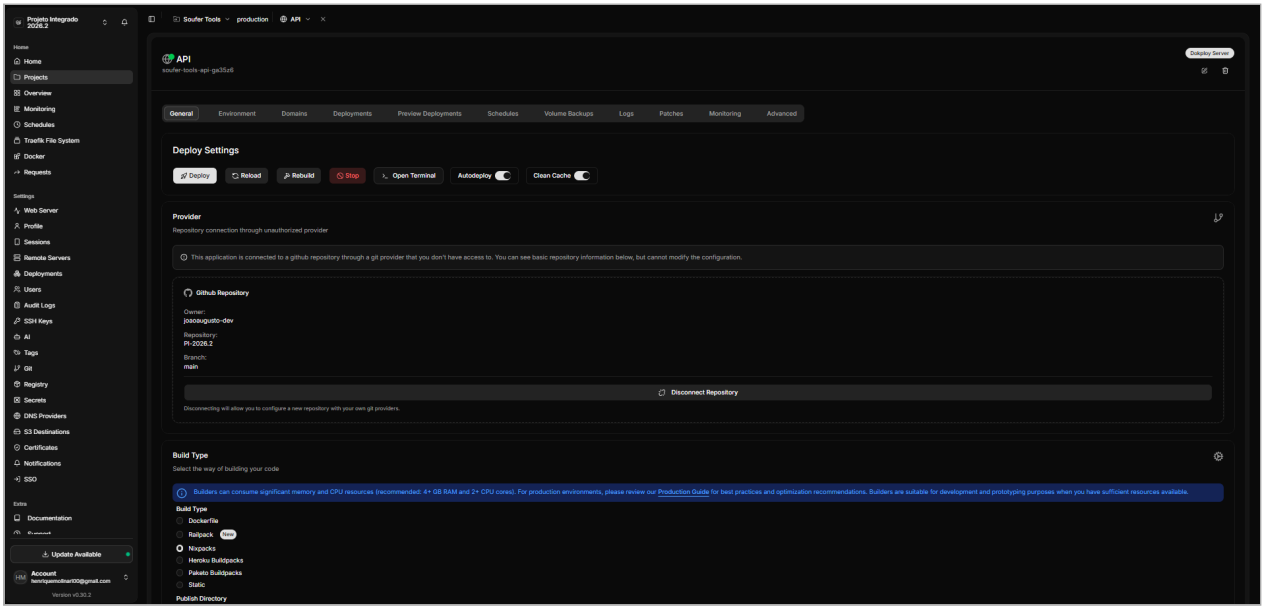


Figura 4 – API de produção: repositório, branch `main` e build Nixpacks.

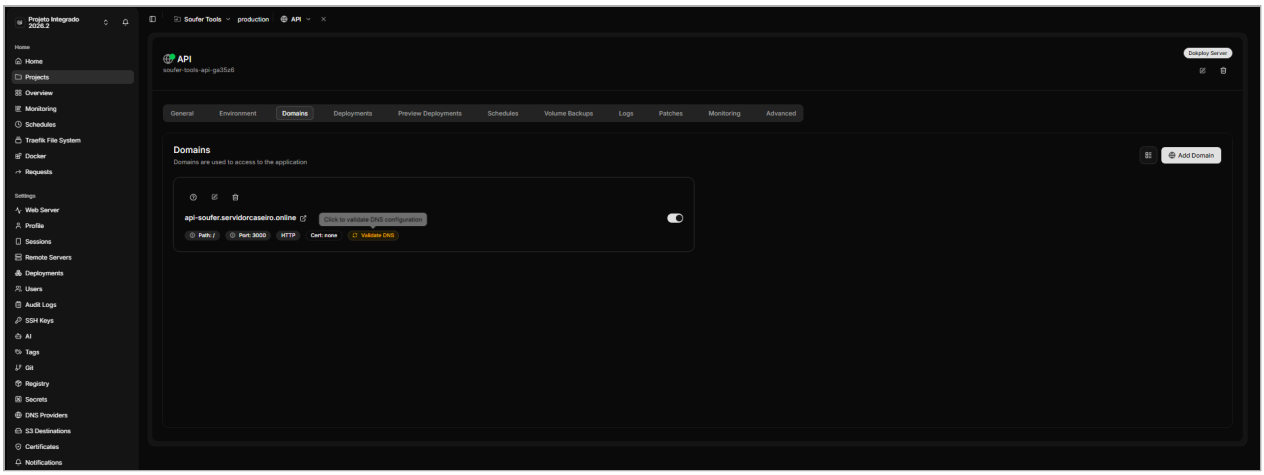


Figura 5 – Domínio da API apontando para a porta 3000.

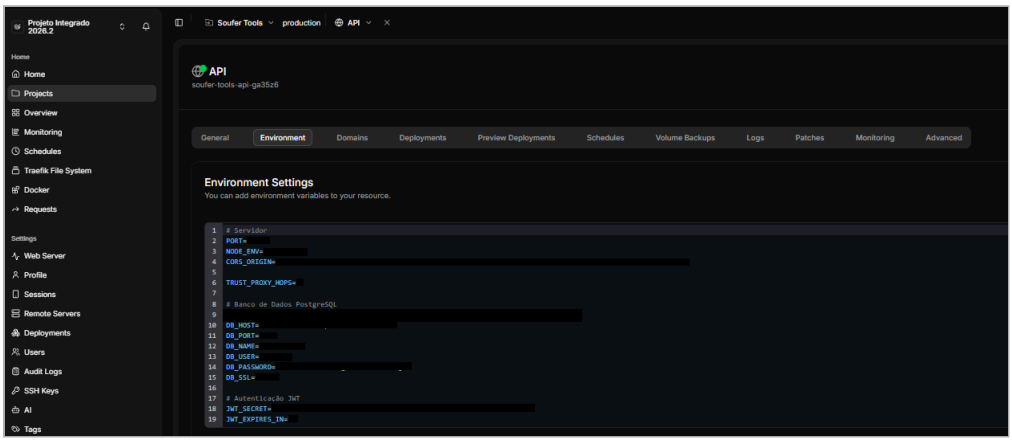


Figura 6 – Variáveis de ambiente da API (valores ocultados).

2.4 Front-end

O front (React com Vite) também é buildado pelo Dokploy e fica no domínio `soufer-tools.servidorcaseiro.online`, com certificado HTTPS do Let's Encrypt emitido pelo Traefik. O endereço da API entra no build pela variável `VITE_API_URL`.

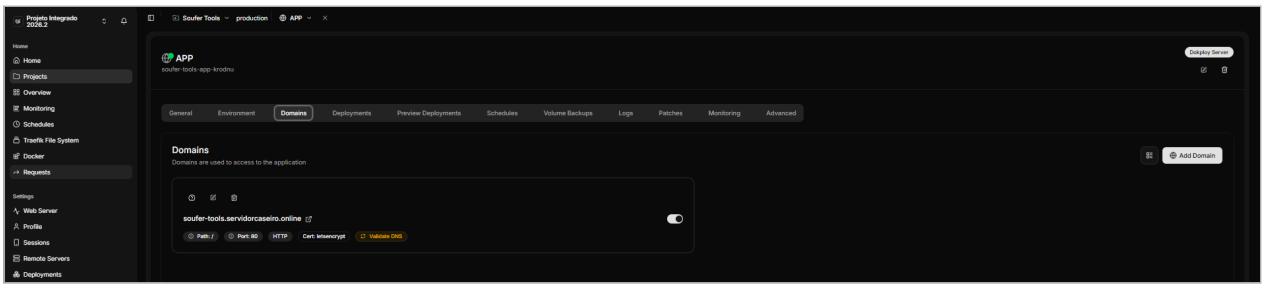


Figura 7 – Domínio do front de produção, com certificado Let's Encrypt.

2.5 Ambiente de desenvolvimento

O ambiente de desenvolvimento tem a mesma estrutura, mas acompanha a branch `development`. Usamos ele para testar as mudanças antes de levar para a produção.

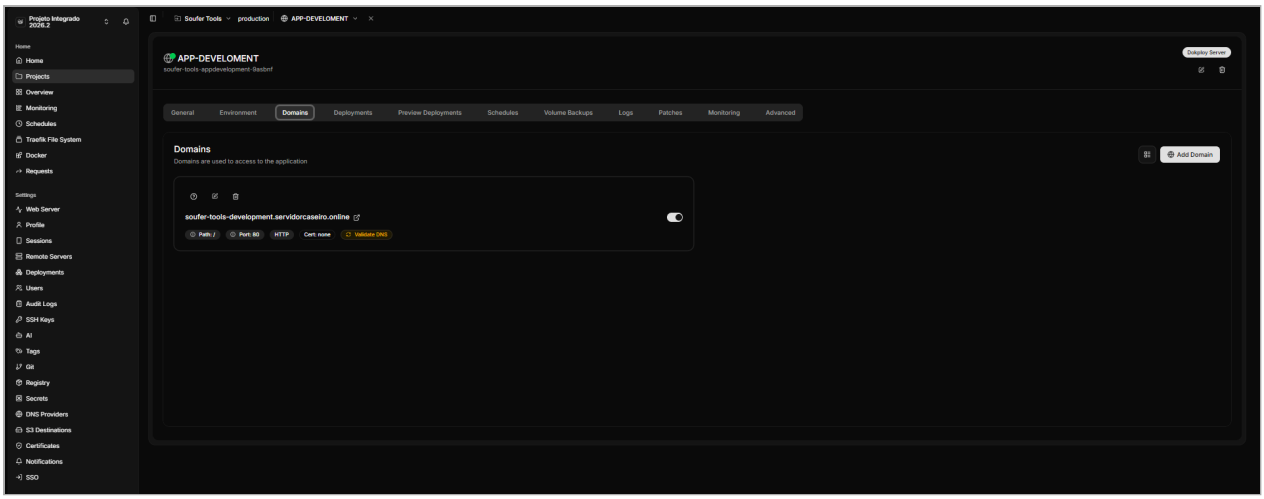


Figura 8 – Domínio do front de desenvolvimento.

3. Testes de comunicação entre os componentes

3.1 API e banco (healthcheck)

A API tem a rota pública `GET /v1/health`, que faz uma consulta de verdade no PostgreSQL. Se o banco responde, retorna 200 com `database.status: "connected"`; se não, retorna 503. Em produção a rota não mostra o nome do banco nem a mensagem de erro do driver. Abrimos a rota pelo navegador, pelo domínio e por HTTPS, em 13/10/2026.

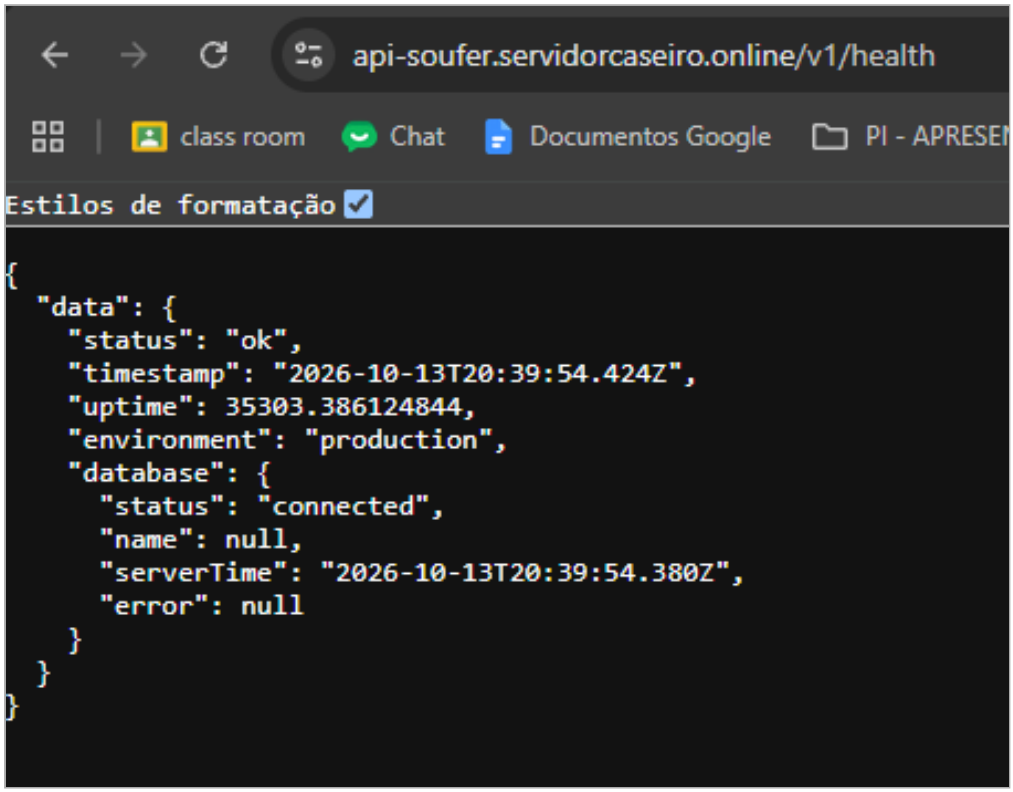


Figura 9 – `/v1/health` em produção: `status: ok` e banco `connected`.

3.2 Front, API e banco juntos

O print abaixo é do sistema em produção, logado como administrador. Os números do dashboard (192 ferramentas cadastradas e disponíveis) vêm do PostgreSQL, passam pela API e aparecem no front, e o indicador "Status API" no topo está verde. Isso mostra a cadeia inteira funcionando: navegador, front, API e banco.

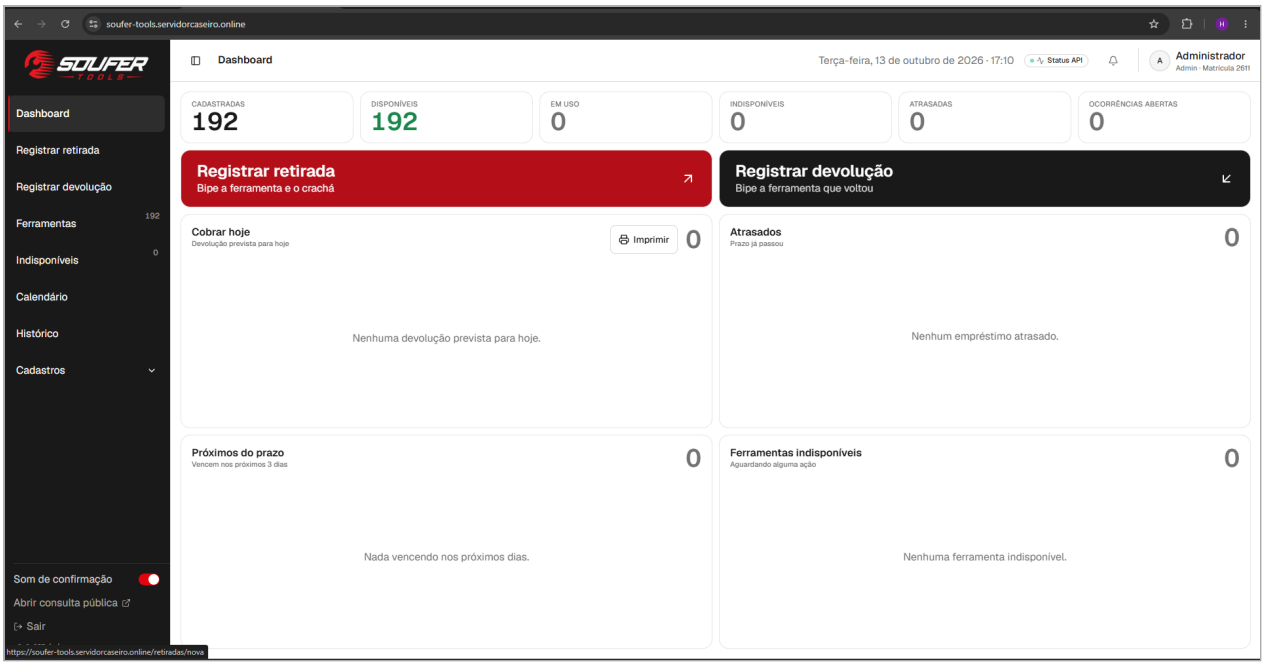


Figura 10 – SOUFER Tools em produção, com dados vindos do banco.

3.3 Consulta direta ao banco

Para conferir os dados na origem, abrimos o terminal do contêiner do banco pelo próprio Dokploy e entramos no `soufer_prod` com o `psql`. O `\dt` lista as tabelas (entre elas `ferramentas`, `colaboradores`, `emprestimos`, `ocorrencias` e `schema_migrations`) e as consultas de contagem retornaram **192 ferramentas** e **113 colaboradores**. O número de ferramentas é o mesmo do dashboard da Figura 10.

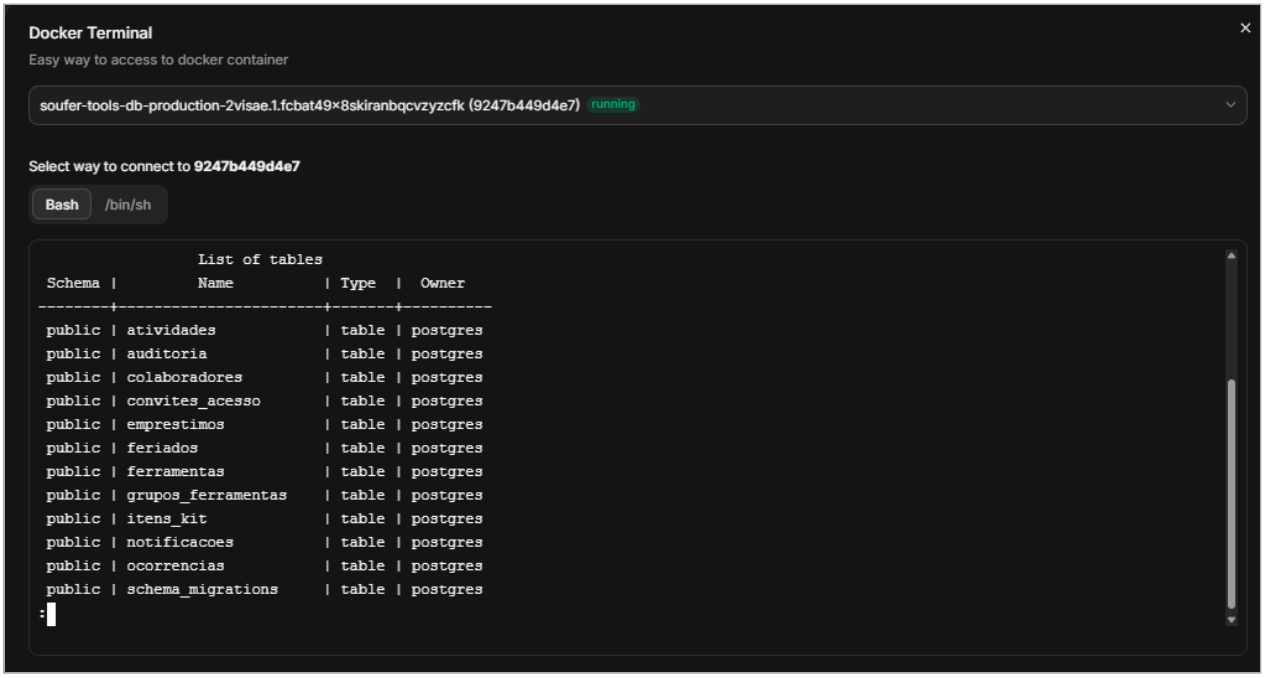


Figura 11 – Tabelas do banco de produção (`\dt`).

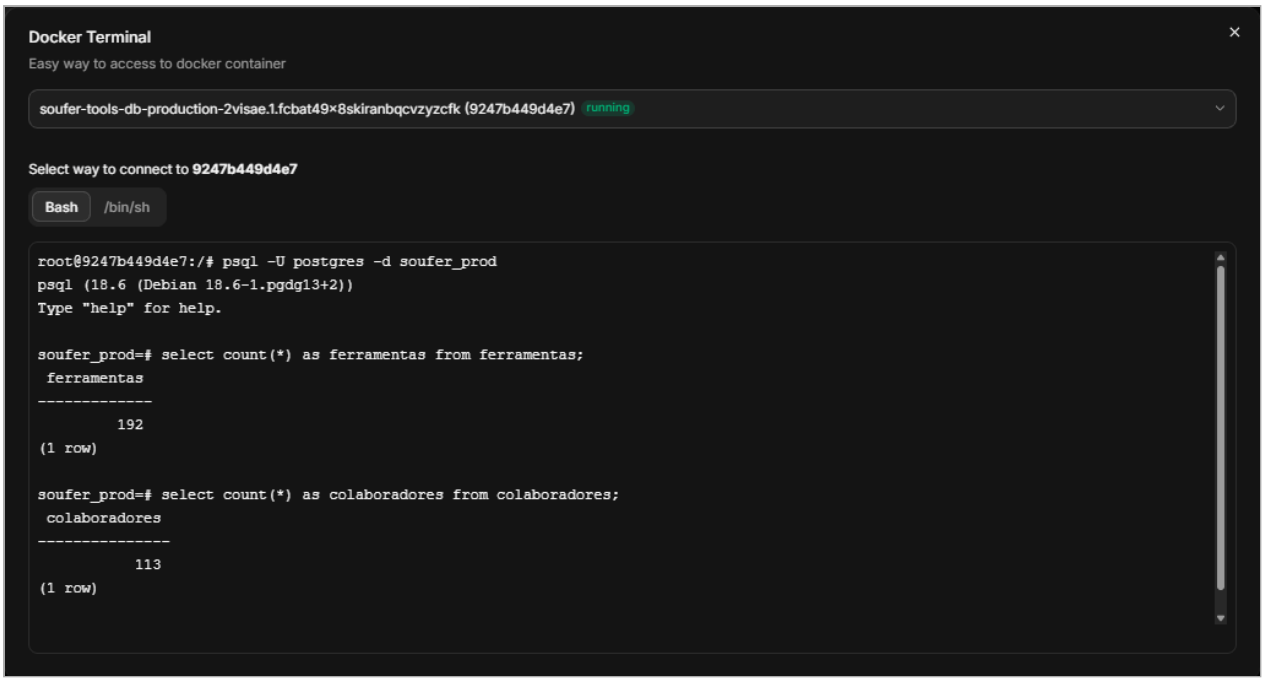


Figura 12 – `select count(*)` em `ferramentas` e `colaboradores`.

4. Logs, monitoramento e deploy

4.1 Logs

A API usa o Pino e grava uma linha em JSON por requisição (método, URL, status e tempo de resposta). Salvamos os logs de produção do dia 13/10/2026 (das 20:02 às 20:39 UTC) no arquivo `logs-api-producao.txt`, em `docs/prints/prints-dokyploy-servidor-caseiro/`. Conferimos o arquivo e ele não tem token, senha nem cabeçalho de autorização. São 99 requisições: 15 com status 200, 1 com 201 (criação de um convite de acesso), 76 com 304 (cache do navegador), 6 com 401 e 1 com 404 (`favicon.ico`).

Os 401 são o comportamento esperado da autenticação: um login com credencial errada foi recusado (401) e o seguinte, correto, passou (200), e as rotas protegidas chamadas sem token também responderam 401. Também aparecem nos logs as chamadas de `/v1/ferramentas`, `/v1/dashboard`, `/v1/notificacoes`, `/v1/emprestimos` e o próprio `/v1/health`.

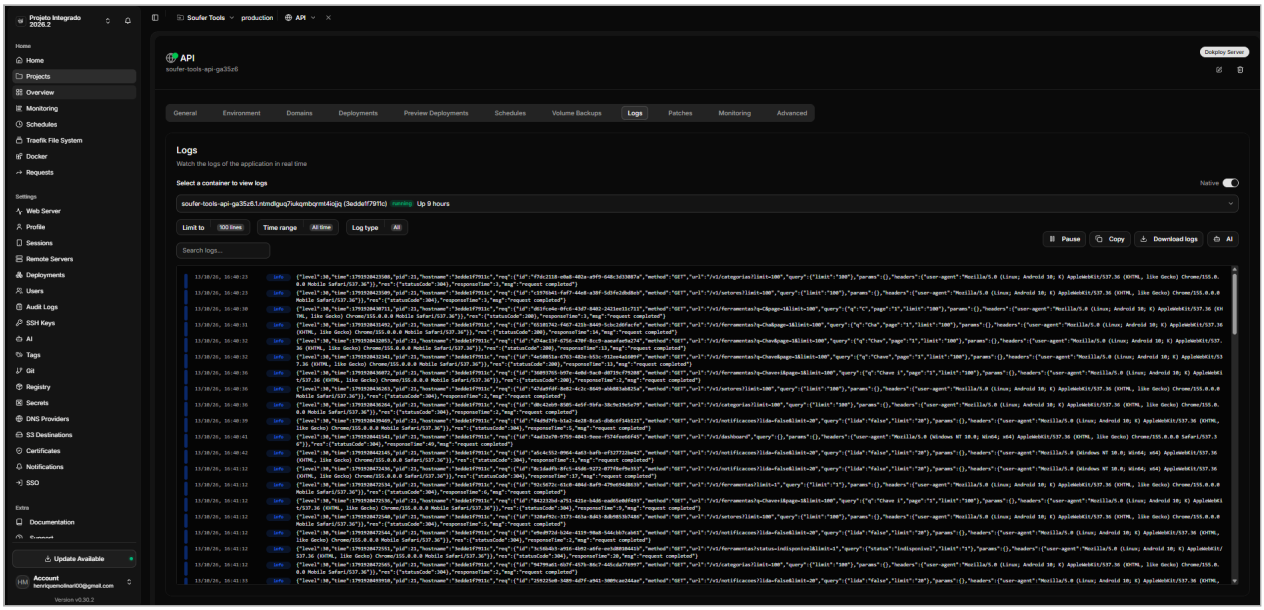


Figura 13 – Logs da API de produção no Dokploy.

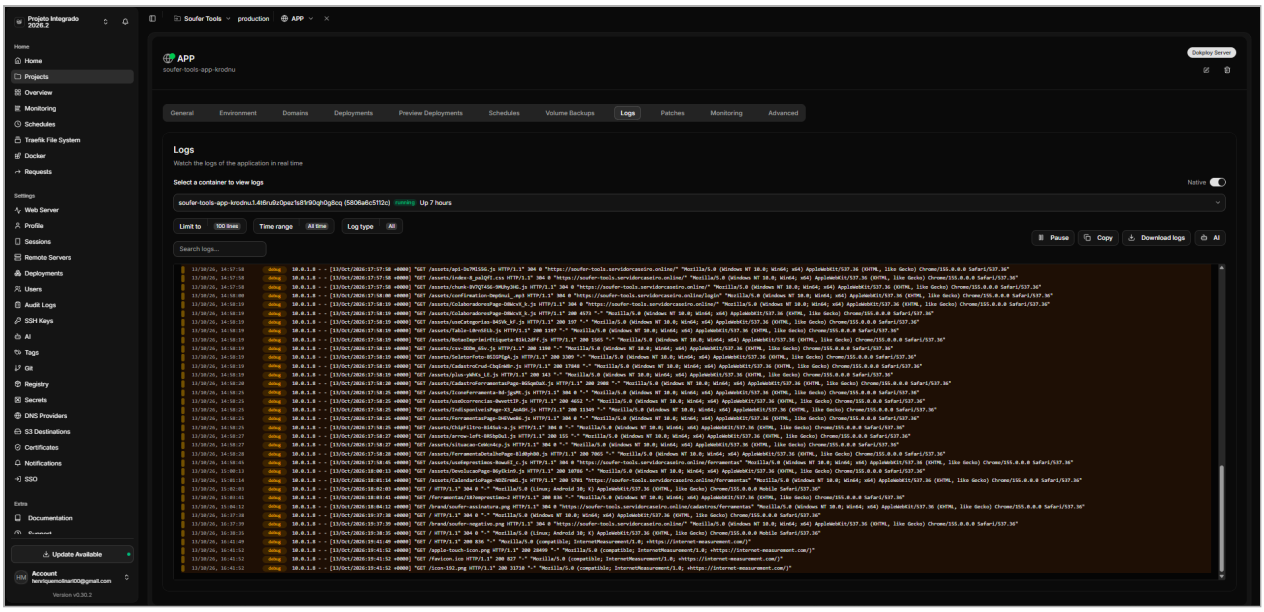


Figura 14 – Logs do front de produção servindo os arquivos da aplicação.

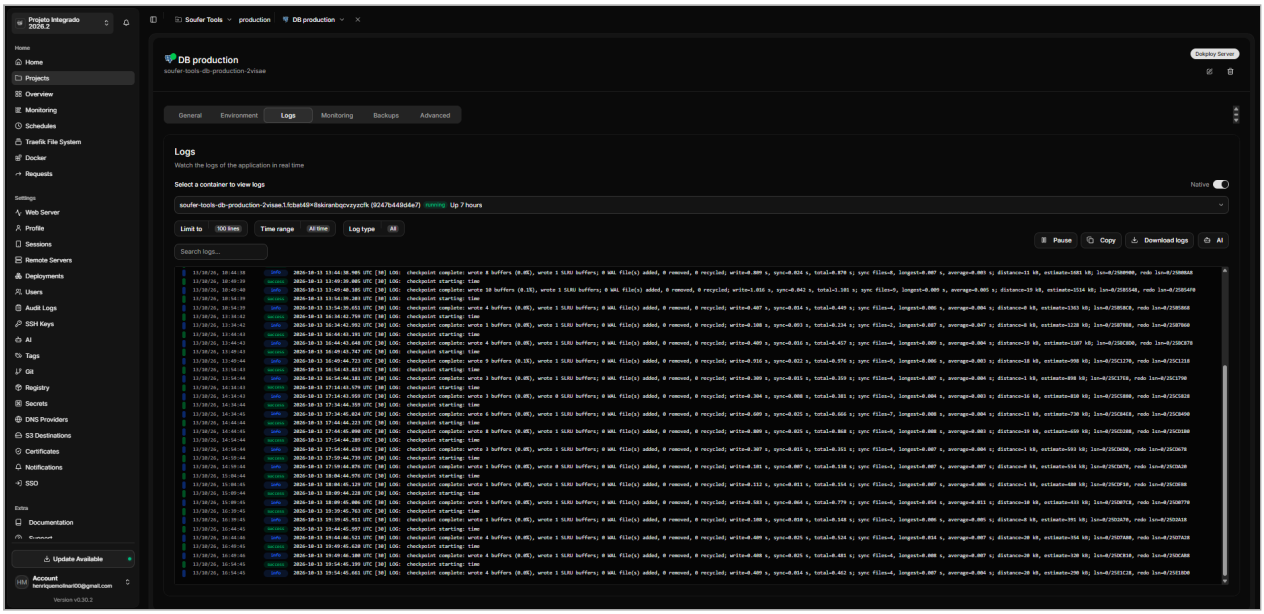


Figura 15 – Logs do PostgreSQL de produção (checkpoints periódicos, sem erros).

4.2 Monitoramento

O Dokploy mostra CPU, memória, disco e rede, do servidor e de cada serviço. Na hora do print o servidor estava com 10,26% de CPU, 6,00 GiB de 7,65 GiB de memória e 98,23 GB de 112,04 GB de disco. Em repouso, a API usava cerca de 45 MiB de memória e o banco cerca de 29 MiB, ou seja, a aplicação é leve.

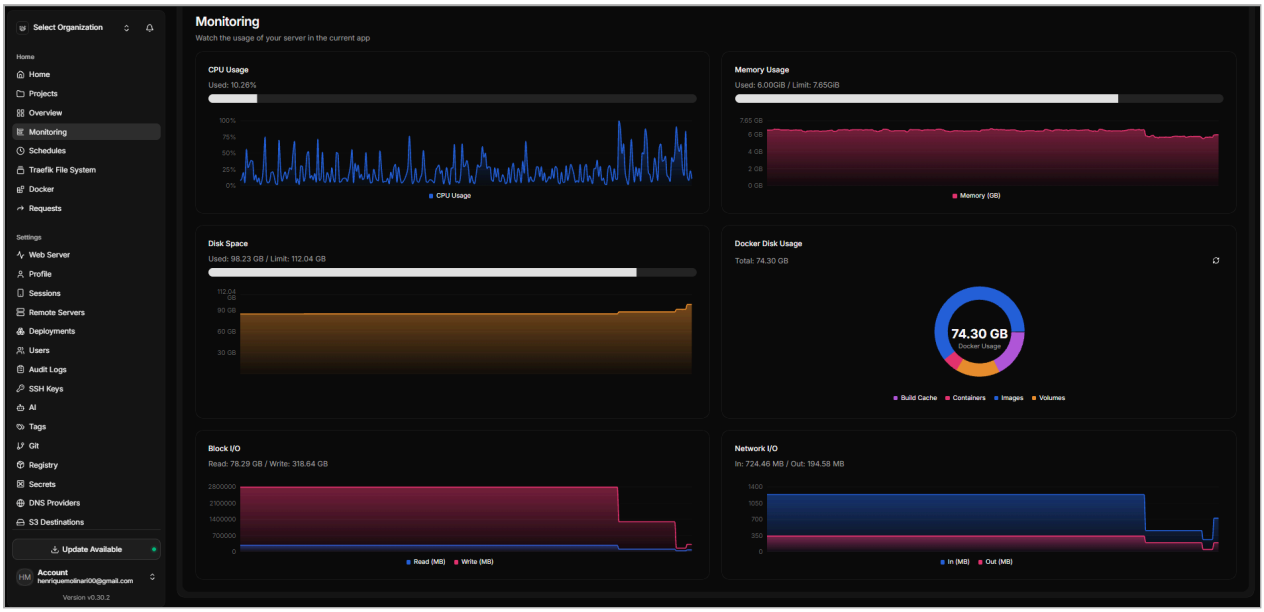


Figura 16 – Monitoramento do servidor.

4.3 Deploy automático

Cada serviço está ligado ao GitHub. Quando um pull request é aceito na `main` (produção) ou na `development` (desenvolvimento), o Dokploy recebe um webhook, baixa o código, gera a imagem com o Nixpacks e troca o contêiner. O histórico guarda o commit, o tempo e o resultado de cada deploy. Cobrimos o endereço do webhook no print, porque ele funciona como uma senha.

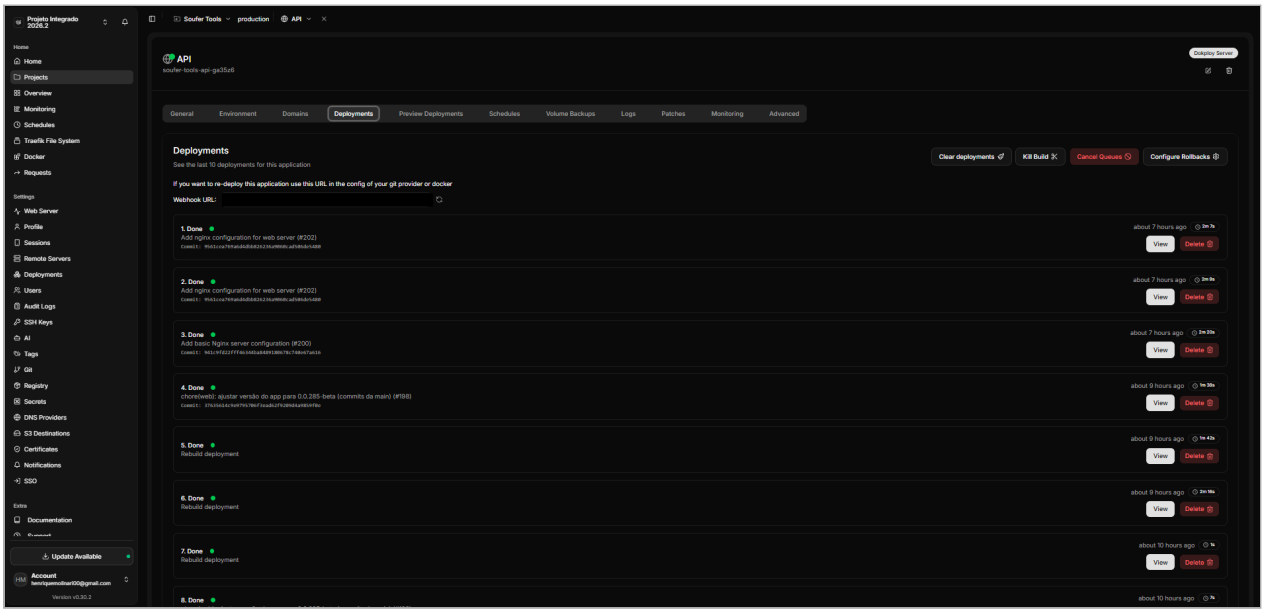


Figura 17 – Histórico de deploys da API de produção.

5. Do servidor próprio para AWS e Azure

O plano de infraestrutura que entregamos em 25/08/2026 (`docs/nuvem/infra-plano.md`) já previa a arquitetura na AWS. O que montamos agora corresponde a ele assim:

- Servidor Linux com Docker (API) corresponde à instância **EC2** (Azure Virtual Machines, GCP Compute Engine).
- PostgreSQL em contêiner corresponde ao **RDS** (Azure Database for PostgreSQL, GCP Cloud SQL).
- Front servido em contêiner corresponde ao **S3 + CloudFront** (Azure Blob Storage + Front Door, GCP Cloud Storage + Cloud CDN).
- Traefik com Let's Encrypt corresponde ao **Nginx + ACM** (ou CloudFront) para o HTTPS.
- Logs e métricas do Dokploy correspondem ao **CloudWatch** (Azure Monitor, GCP Cloud Monitoring).

Quando tivermos acesso, o plano é: (1) criar a EC2, o RDS, o bucket S3 e o CloudFront; (2) subir a API com Nginx e PM2 em fork mode com 1 instância (o rate limit guarda os contadores em memória) e `TRUST_PROXY_HOPS=1`; (3) rodar as migrations no RDS e restaurar os dados do banco atual com `pg_dump` e `pg_restore`, conferindo as contagens (192 ferramentas e 113 colaboradores); (4) publicar o front com a nova `VITE_API_URL`; (5) trocar o DNS só depois de validar o `/v1/health` e um fluxo completo de retirada e devolução, mantendo o servidor atual de reserva. Como a aplicação só depende de variáveis de ambiente, a migração não exige mudar o código. Pela estimativa de custos que fizemos, a AWS continua sendo a opção mais barata (US$ 29,51 por mês, contra US$ 32,52 no GCP e US$ 36,51 na Azure).

6. Pendências

Antes de dar a etapa por encerrada, ainda temos estes pontos para resolver:

- **Backup do banco:** ainda não configuramos um destino de backup no Dokploy (Figura 18). Vamos configurar um destino S3 e agendar um `pg_dump` diário, testando a restauração.
- **Porta externa do banco:** o serviço do banco de produção está com a porta externa 5432 preenchida. Vamos remover e deixar o acesso só pela rede interna.
- **Banco de desenvolvimento:** o `soufer_dev` ainda não recebeu todas as migrations (os logs de desenvolvimento reclamam da tabela `usuarios`). Falta rodar `npm run db:migrate` nesse serviço. A produção não é afetada.
- **Servidor único:** hoje tudo roda em uma máquina só, sem redundância. Na nuvem vamos incluir alarmes de saúde e uma instância de reserva.

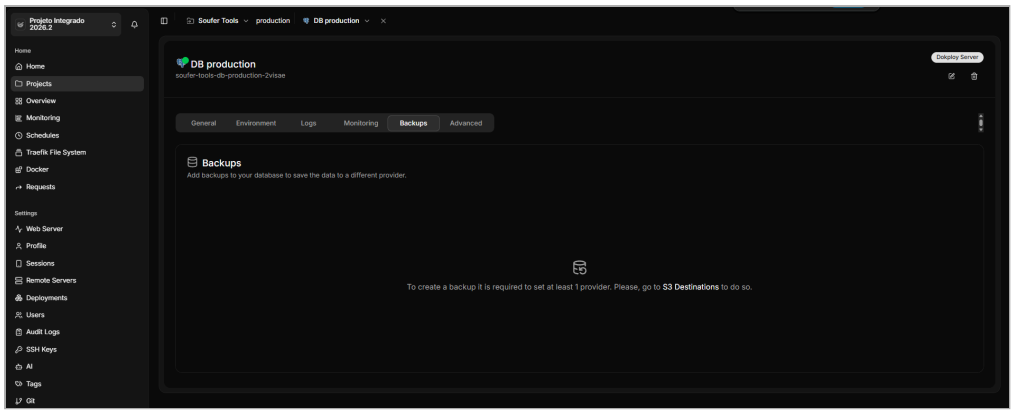


Figura 18 – Aba de backups do banco de produção, ainda sem destino configurado.

7. Resultado

Montamos em um servidor nosso o ambiente com suporte a Node.js e o banco PostgreSQL acessível, com produção e desenvolvimento, domínios com HTTPS, deploy automático pelo GitHub, logs e monitoramento. Os testes mostram a comunicação entre os componentes funcionando em produção: o healthcheck da API confirma a conexão com o banco, o dashboard exibe dados lidos do PostgreSQL, as consultas feitas direto no banco batem com a tela e os logs registram login e rotas protegidas se comportando como esperado.

Como não temos créditos nem acesso à AWS e à Azure, esta entrega é uma solução paliativa, escolhida para não pôr em risco um sistema que já está em uso. Assim que o sistema estabilizar e o acesso for liberado, vamos refazer a implantação nessas plataformas seguindo o plano da seção 5 e entregar as evidências. Os guias que usamos estão no repositório, em `docs/nuvem/dokploy.md` (passo a passo da implantação) e `docs/nuvem/custos-nuvem.md` (comparativo de custos).